



Python应用基础

第8章 面向对象编程

陈刚

中山大学岭南学院

提高篇目录

Contents

第 8 章

面向对象编程

第 9 章

异常处理

第 10 章

数据库应用

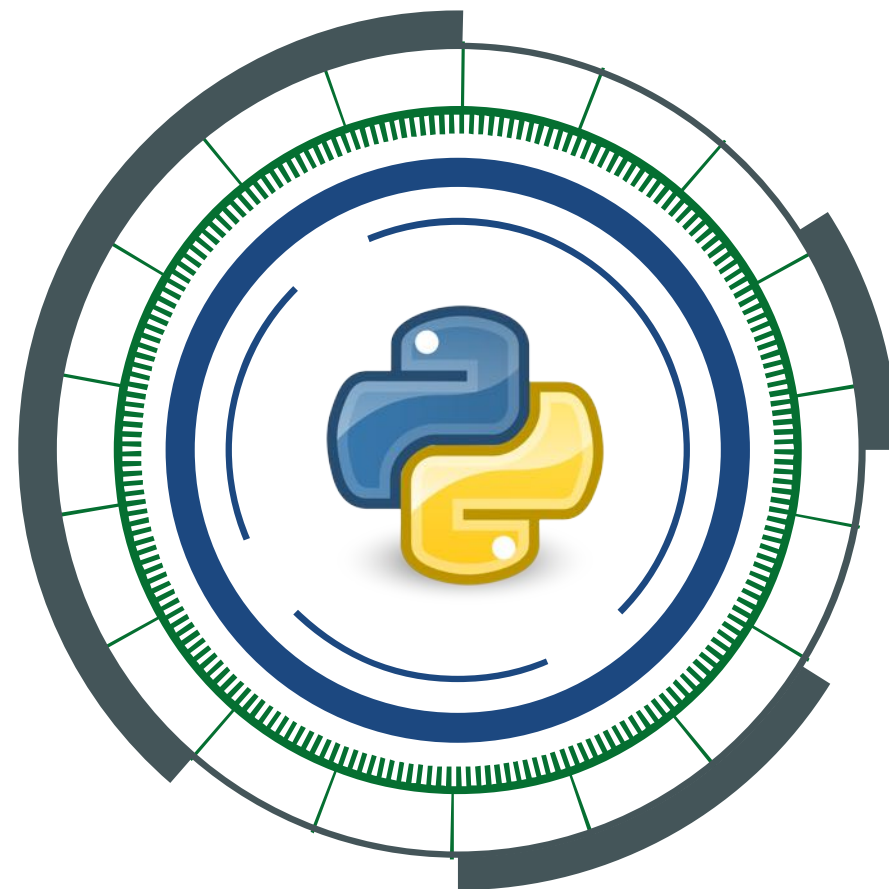


pythonTM
programming



学习目标

- 1 熟悉面向对象程序设计的概念
- 2 掌握类的定义和使用方法
- 3 熟悉实例属性和类属性的概念和区别
- 4 掌握类方法的概念
- 5 了解面向对象中继承、多态和封装的概念



编程语言分类——按执行方式分类

- 代码是怎样跑起来的

类型	工作原理	特点	代表语言
编译型 (Compiled)	运行前先将全部代码翻译成二进制文件	运行速度快，但开发调试慢	C, C++, Go, Rust
解释型 (Interpreted)	运行时一行一行翻译并执行	开发效率高，跨平台好，但速度慢	Python, Ruby, PHP
混合型 / 字节码	先编译成中间码，再由虚拟机运行	兼顾性能与跨平台	Java, C#

编程语言分类——按编程范式分类

- 解决问题的思路，决定了如何组织代码逻辑
- 几乎所有编程语言都归属于其中一种或多种的组合

范式	核心思想	典型语言	关键特征
命令式编程 Imperative	以指令序列描述“如何做”，通过改变程序状态来推进计算	C, Pascal, Fortran	变量赋值、循环、状态可变
函数式编程 Functional	视计算为数学函数的求值，强调无副作用和不可变数据	Haskell, Scheme, ML	纯函数、递归、惰性求值
逻辑式编程 Logic	基于谓词逻辑，声明事实和规则，由系统推理求解	Prolog, Datalog	事实/规则库、回溯、模式匹配
面向对象编程 Object-Oriented	通过“对象”封装数据和行为，以消息传递实现交互	Smalltalk, Java, C++	封装、继承、多态

编程语言分类——按编程范式分类

- 命令式：你是个一丝不苟的车间主任，指挥工人一步步拧螺丝
- 函数式：你是流水线设计师，把原材料扔进管道，出口就是成品
- 逻辑式：你是个高冷的数学家，甩给助手一句话：“解出 x 满足……”，等结果即可
- 面向对象：你是CEO，把任务分给几个能力完备的部门经理（对象），他们各自搞定

范式	核心关注点	代码风格	状态管理
命令式编程	步骤序列	循环、判断、赋值	显式可变变量
函数式编程	数据变换管道	函数组合、无副作用	不可变，数据流经函数
逻辑式编程	事实与规则	声明关系，系统推理	隐式（回溯存储）
面向对象编程	对象交互	调用方法、消息传递	对象内部封装

以python为例，对比四种范式

- 判断学生考试是否“通过”（及格线为60分）
- 数据：[45, 72, 88, 59, 90]
- 期望输出：[72, 88, 90]（即所有 ≥ 60 分的学生分数）

```
scores = [45, 72, 88, 59, 90]
```

```
# === 1. 命令式 (Imperative) =====  
# 思维：一步一步告诉计算机怎么做——初始化、循环、判断、累加  
def imperative_get_passing(scores):
```

```
    result = []  
    for score in scores:  
        if score >= 60:  
            result.append(score)  
    return result
```

```
# === 2. 函数式 (Functional) =====  
# 思维：数据经过一系列变换——过滤 → 收集结果
```

```
def functional_get_passing(scores):  
    return list(filter(lambda x: x >= 60, scores))
```

```
# 更 Pythonic 的函数式（列表推导式）
```

```
def functional_pythonic_get_passing(scores):  
    return [score for score in scores if score >= 60]
```

```

# ===3. 逻辑式 (Logic Programming Style) =====
# 思维: 声明“什么是及格分数”的逻辑规则, 然后问系统“哪些 X 满足条件?”
# 注意: Python 原生不支持逻辑式, 这里用生成器模拟“按需回溯”的感觉
def logic_style_get_passing(scores):
    # “规则”: 及格分数 X 是列表中的一个元素, 且 X >= 60
    def passing_score_generator():
        for x in scores:      # 模拟“member(X, List)”
            if x >= 60:        # 模拟“X >= 60”
                yield x        # 回溯产生下一个满足条件的值

    return list(passing_score_generator())

# ===4. 面向对象 (Object-Oriented) =====
# 思维: 封装数据和行为, 通过对象发送消息来完成任任务
class ScoreProcessor:
    def __init__(self, scores):
        self._scores = scores    # 封装数据

    def get_passing(self, threshold=60):
        result = []
        for score in self._scores:
            if score >= threshold:
                result.append(score)
        return result

```

```

# === 执行与输出 =====
if __name__ == "__main__":
    print("原数据:", scores)
    print("-" * 50)
    print(f"命令式结果: {imperative_get_passing(scores)}")
    print(f"函数式结果: {functional_get_passing(scores)}")
    print(f"逻辑式风格结果: {logic_style_get_passing(scores)}")

    processor = ScoreProcessor(scores)
    print(f"面向对象结果: {processor.get_passing()}")

```




8.1

定义和使用类

定义和使用类

- 面向对象编程（object-oriented programming, OOP）是当今使用最广泛的编程技术，几乎所有对象都提供了创建和管理对象的方法。
- 类和对象构成了 Python 编程语言的核心功能。
- 将问题中涉及的现实对象抽象出共同特征，定义新的种类，称为类（class）。
- 通过类，实例化现实世界中的对象。

类：蓝图/模板，比如"汽车设计图"

对象：具体的实例，比如"一辆红色的宝马"

定义和使用类

- 使用 class 关键字定义类:

```
class name:  
class body
```

- 定义类后，与其他内置类型一样，可以利用该类实例化对象。

实例化的方法，与函数调用类似。

- 创建实例化对象后， 可以通过赋值变量名的方式为实例添加属性：

```
instance_name.attr_name = value
```

定义和使用类

【例】定义一个新的类 Person 来表示人员信息。并用 Person 类实例化对象。

然后为对象添加姓名、部门和电子邮箱等属性。

```
❶ class Person:
❷     pass

❸ zhangsan = Person()
❹ print(type(zhangsan))

<class '__main__.Person'>

❺ zhangsan.name = '张三'
❻ zhangsan.department = '管理'
❼ zhangsan.email = 'zhangsan@example.com'
❽ print(zhangsan.name)

张三
```

定义和使用类

- 内置函数 `isinstance()` 用来测试一个对象是否属于某一个类的实例：

```
❶ isinstance(zhangsan, Person)  
True
```

- 在 Python 中，所有类都是基于一个叫作 `object` 的类，在 `object` 类的基础上进行扩展，被称为**继承**。因此，所有对象都是 `object` 类的实例

```
❷ isinstance(zhangsan, object)  
True
```

课堂练习1：抓住那个对象

场景：程序里的 Bug 会伪装成各种类型的对象。你需要识别它们的身份。

任务：

- 定义一个 Monster（怪兽）类，里面只写一行 pass。
- 创建两个实例：bug_a = Monster() 和 bug_b = [1, 2, 3]。
- 使用 isinstance() 编写一个函数 detect(obj)，如果 obj 是 Monster 的实例，打印“🔍 发现 Bug! 快跑! ”，否则打印“✅ 安全，这是个正常对象。”
- 挑战：试试 detect(bug_a) 和 detect(bug_b)。



8.2

属性

属性（成员变量）

- 属性通常用来存取数据，分为实例属性和类属性两种。

实例属性：每个对象独有的数据

类属性：所有对象共享的数据

```
class Student:
    school = "第一中学" # 类属性

    def __init__(self, name, age):
        self.name = name # 实例属性
        self.age = age   # 实例属性
```


实例属性

- 通过给“实例名.属性名”赋值，为实例对象创建属性，称为实例属性。

```
❶ zhangsan.idCard = '510105199510080017'  
❷ print(zhangsan.idCard)  
510105199510080017
```

- 对于实例属性来说，每个实例拥有自己单独的属性值，实例属性只能通过实例名称来访问：

```
❶ lisi = Person()  
❷ lisi.idCard = '510105199606080021'  
❸ print(lisi.idCard)  
510105199606080021
```

类属性

- 在定义类时，也可以在类体中通过赋值语句创建数据属性，称为类属性。
- 类属性可以用来管理通过该类创建的所有实例的属性信息。
- 与实例属性不同，由同一个类创建的所有实例公用同一个类属性。
- 可以通过类名称来修改它，通过实例名称或类名称来引用它。

类属性

【例】定义类 Person，其中包含类属性 city，利用 Person 类创建实例，并分别设置实例属性和类属性值。

```
❶ class Person:  
    city = '成都'  
  
❷ zhangsan = Person()  
❸ zhangsan.name = '张三'  
❹ print(zhangsan.name, zhangsan.city)
```

张三 成都

```
❺ lisi = Person()  
❻ lisi.name = '李四'  
❼ print(lisi.name, lisi.city)
```

李四 成都

```
❽ Person.city = '上海'  
❾ print(zhangsan.name, zhangsan.city)  
❿ print(lisi.name, lisi.city)
```

张三 上海
李四 上海

类属性

【例续】 如果使用实例名称为类属性赋值，实际上会产生一个同名的实例属性，这样对其他实例没有影响。

```
❶ zhangsan.city = '北京'  
❷ print(zhangsan.name,  
❸ zhangsan.city)  
print(lisi.name, lisi.city)
```

```
张三 北京  
李四 上海
```

课堂练习2：记忆面包

考试前，全班同学（实例）都吃了“记忆面包”，但配方（类属性）被老师改了两次

```
class Student:
    bread_formula = "面粉" # 类属性：记忆面包配方

    def __init__(self, name):
        self.name = name

# 老师第一节课给的配方
print(f"全班同学吃的是面包配方：{Student.bread_formula}")

student_a = Student("张三")
student_b = Student("李四")
```

```
# 张三偷偷改了自己的配方（实例属性赋值）
student_a.bread_formula = "巧克力"
print(f"{student_a.name} 记得配方是：{student_a.bread_formula}")
print(f"{student_b.name} 记得配方是：{student_b.bread_formula}")

# 老师第二节课改了全班配方（修改类属性）
Student.bread_formula = "草莓"
print(f"老师改配方后，{student_b.name} 记得：{student_b.bread_formula}")
print(f"老师改配方后，{student_a.name} 记得：{student_a.bread_formula}? 为什么? ")
```



8.3

方法

方法概述

- 方法为对象提供行为，实际上就是一些类体中的函数，称为方法。
- 有两种方式可以调用方法：一种是通过“类名.方法”的形式，另一种是通过“实例.方法”的形式。

【例】 对于字符串类str，分别利用“类名.方法”和“实例.方法”的形式调用upper()方法。

```
❶ print(str.upper('Fintech'))  
❷ print('Fintech'.upper())
```

```
FINTECH  
FINTECH
```

为类添加方法

- 方法和函数的工作方式是一致的，不同在于：方法的第一个参数总是方法调用的实例对象。在类体中定义方法时，第一个参数通常为 `self`。
- 在调用方法时，不需要显示给 `self` 赋值，Python 会**自动将实例赋值给 `self`**，然后再依次将调用的**实参赋值给 `self` 参数后的形参**。
- 在方法体中，可以通过 `self` 来访问或修改当前实例的属性。

```
class Student:
    def __init__(self, name):
        self.name = name # 要点3: 通过self访问实例属性

    def introduce(self, greeting):
        # 要点1: self是第一个参数, 指向实例对象
        # 要点2: 调用时无需手动传self, Python自动处理
        print(f'{greeting}, 我是{self.name}')
        # 要点3: 通过self修改/访问属性

# 创建实例
zhang = Student("张三")

# 调用方法时, 只需要传greeting参数
zhang.introduce("大家好")
# Python自动将zhang传给self
# 等价于: Student.introduce(zhang, "大家好")
```


为类添加方法

【例】 定义类 Person，其中包含方法 introduce，需要传入一个问候语 message。该方法在打印出问候语后，将打印出当前实例的 name、department 和 email 属性信息。

```
① class Person:
②     def introduce(self, message):
③         print(message)
④         print('我是 {} \n在 {} 部门工作, 邮箱 {}'.format(self.name, self.department,
⑤ self.email))
⑥ zhangsan = Person()
⑦ zhangsan.name = '张三'
⑧ zhangsan.department = '管理'
⑨ zhangsan.email = 'zhangsan@example.com'
⑩ zhangsan.introduce('大家好')
```

大家好
我是张三
在管理部门工作, 邮箱 zhangsan@example.com

如果使用“类名.方法”的形式调用 introduce 方法，则需要给出第一个参数 self 的值。

```
⑩ Person.introduce(zhangsan, '大家好')
```

大家好
我是张三
在管理部门工作, 邮箱 zhangsan@example.com

为类添加方法

【例】 定义类 Person，通过 gender() 和 birthday() 方法分别获取性别与生日信息

```
① class Person:
②     def gender(self):
        flag = int(self.idCard[-2])
        return '男' if flag % 2 == 1 else '女'
③     def birthday(self):
        year = self.idCard[6:10]
        month = self.idCard[10:12]
        day = self.idCard[12:14]
        return '{} 年 {} 月 {} 日 '.format(year, month, day)

④ zhangsan = Person()
⑤ zhangsan.idCard = '510105199510080017'
⑥ print('性别: ', zhangsan.gender())
⑦ print('生日: ', zhangsan.birthday())
```

性别: 男

生日: 1995 年 10 月 08 日

利用构造方法初始化

- 通常使用一个特定的“__init__”方法为实例的属性赋值，这个方法称为构造方法。
- 构造方法会在每次创建一个实例时自动运行。

```
① class Person:  
②     def __init__(self, name, dept, p_email, p_idCard):  
③         self.name = name  
④         self.department = dept  
⑤         self.email = p_email  
⑥         self.idCard = p_idCard
```

利用构造方法初始化

- 在创建实例时，使用“类名”（参数列表）的形式，将实例的姓名、部门、电子邮箱和身份证号码作为实参传递给构造方法：

```
❶ zhangsan = Person('张三', '管理', 'zhangsan@example.com', '510105199510080017')  
❷ print(zhangsan.name)
```

张三

利用构造方法初始化

- 在定义构造方法时，可以为参数设置默认值，这样就可以在创建实例时，省略一些实参：

```
① class Person:
②     def __init__(self, name, dept='人事', p_email='', p_idCard=''):
        self.name = name
        self.department = dept
        self.email = p_email
        self.idCard = p_idCard

③ zhangsan = Person('张三', '管理')
④ print(zhangsan.name, zhangsan.department)
⑤ lisi = Person('李四')
⑥ print(lisi.name, lisi.department)
```

张三 管理
李四 人事

课堂练习3：自我介绍机器人

场景：你作为“班级点名机器人”，需要创建一个 Student 类，让每个新生做自我介绍，并统计当前“已报到”的人数。

任务：

- 定义一个 Student 类。
- 在 `__init__` 中，每初始化一个学生，就打印一个随机（或固定）的二次元台词，比如“我是{name}，驾到！通通闪开！”。
- 给 Student 添加一个类属性 `count = 0`，在 `__init__` 中让其自增。
- 添加一个方法 `say_hello`，里面只写一句 `print(f'{self.name}: 其实我社恐...' + "😓")`。
- 创建两三个学生实例，分别调用他们的 `say_hello`。

更多特殊方法

- 在 Python 中，除了构造方法外，类还支持大量的特殊方法。和构造方法“__init__”一样，这些特殊方法都是以两条下划线“__”开头和结束的。
- 通过内置函数 dir()，可以看到类中包含的方法：

```
❶ print(dir(Person))
```

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__',  
 '__ge__', '__getattr__', '__gt__', '__hash__', '__init__', '__init_subclass__',  
 '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__',  
 '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__']
```

这些方法大部分来自基类 object。通过重写这些特殊方法，可实现更多功能

更多特殊方法

- 对于 Person 类创建的实例对象 zhangsan，在调用 Python 内置函数 print() 进行打印时，得到的是 zhangsan 对象的内存地址：

```
❶ zhangsan = Person('张三', '管理')  
❷ print(zhangsan)  
  
<__main__.Person object at 0x7f8ed02ba710>
```


更多特殊方法

- 重新定义“__str__”方法可以改变这个返回的结果，当利用 Python 内置函数 print() 进行打印时，对象的“__str__”方法会自动被调用：

```
❶ class Person:
❷     def __init__(self, name, dept='人事', p_email='', p_idCard=''):
        self.name = name
        self.department = dept
        self.email = p_email
        self.idCard = p_idCard
❸     def __str__(self):
        return '姓名 :{}, 部门 :{}'.format(self.name, self.department)

❹ zhangsan = Person('张三', '管理')
❺ print(zhangsan)
```

姓名 :张三, 部门 :管理

更多特殊方法

- 对“__eq__”方法进行重写，可改变相等测试的结果。与“__str__”不同的是，该方法除了当前实例 self 外，还需要另外传入一个实例 other，以便进行比较：

```
❶ class Person:
❷     def __init__(self, name, dept='人事', p_email='', p_idCard=''):
        self.name = name
        self.department = dept
        self.email = p_email
        self.idCard = p_idCard
❸
❹     def __eq__(self, other):
        return self.name == other.name

❺ zhangsan = Person('张三', '管理')
❻ zhangsan2 = Person('张三', '人事')
❼ print(zhangsan == zhangsan2)

True
```

课堂练习4：两只烤鸭长得一样吗

场景：餐厅有两盘烤鸭，但它们来自不同厨师（不同实例）。

任务：

- 定义 RoastDuck 类，含有 weight, price 属性。
- 不定义 __str__，打印实例时显示 <__main__.RoastDuck object at ...>。问学生：“这看起来像烤鸭吗？像一坨乱码。”
- 定义 __str__(self)，让它返回 f" 一只 {self.weight}kg 的烤鸭，售价 {self.price} 元”。
- 不定义 __eq__，比较 duck1 == duck2（属性相同）结果 False。问：“为什么明明一样的鸭子，它们不是同一只？”
- 定义 __eq__(self, other)，如果重量和价格相同则返回 True。再比较一次。



8.4

高级话题

继承

- 通过内置函数 `dir()` 查看自定义 `Person` 类的方法时，可以看到，`Person` 类中绝大部分方法都没有在类中定义，这些方法来自 Python 中所有类的基类 `object`。这就是面向对象编程中的继承 (inheritance)。
- 利用继承，可以把一些通用属性和方法代码实现一次，在设计新类时，不再需要重写这些代码就可以使用这些通用属性和方法。
- 通过继承，所有的类形成一种层次结构。在这种层次结构中，被继承的类称为父类或基类，新的类称为子类或派生类。

继承

【例】 在人员信息管理中，人员类型分为学生和老师。他们有共同的属性，例如姓名和电子邮件。不同之处在于，老师还有职工号、部门和授课课程，学生有学号、所在班级和课程列表等。用代码定义父类 Person 和两个子类 Student 和 Teacher，以便实现人员管理。

继承

```
❶ class Person:
    def __init__(self, name, email=''):
        self.name = name
        self.email = email

❷ class Student(Person):
    def __init__(self, name, email, stu_no, class_name, courses):
❸         super().__init__(name, email)
        self.stu_no = stu_no
        self.class_name = class_name
        self.courses = courses.copy()

❹ class Teacher(Person):
    def __init__(self, name, email, teacher_no, dept, course):
❺         super().__init__(name, email)
        self.teacher_no = teacher_no
        self.department = dept
        self.course = course

❻ zhangsan = Student('张三', 'zhangsan@example.com', '1008', '十班', ['程序设计', '金融数据分析'])
print(zhangsan.name, zhangsan.email, zhangsan.stu_no, zhangsan.class_name)
print(zhangsan.courses)

❼ teacher_li = Teacher('李老师', 'li@example.com', 'T1001', '金融学院', '金融数据分析')
print(teacher_li.name, teacher_li.email, teacher_li.teacher_no, teacher_li.department)
print(teacher_li.course)
```

张三 zhangsan@example.com 1008 十班

['程序设计', '金融数据分析']

李老师 li@example.com T1001 金融学院

金融数据分析

继承

- `super()`
→ 调用父类方法（保留父类功能）
- 重写
→ 子类重新定义方法（覆盖父类功能）

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "动物叫声"

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name) # 调用父类方法
        self.breed = breed

    def speak(self): # 方法重写
        return "汪汪汪"

# 测试
dog = Dog("旺财", "金毛")
print(dog.speak()) # 汪汪汪 (重写生效)
print(dog.name)   # 旺财 (super()调用了父类init)
```


多态

- 父类的方法在子类中也可以被改写。这使得父类同一个方法在不同的子类中具有不同的形态。这就是面向对象编程中的多态 (polymorphism)。
- 对于同一个表达式，利用多态的特性，可以实现基于不同类型对象的不同操作。例如，对于下列表达式：

$a + b$

当 a 和 b 的数据类型是整数或浮点数时，实现的是算术运算的加法；

当 a 和 b 是字符串、列表或元组时，实现的是字符串、列表或元组的连接。

多态

【例】为 Person 类添加 introduce 方法，用于实现人员的自我介绍。在 Student 类和 Teacher 类中改写这个方法。

多态

```
❶ class Person:
    def __init__(self, name, email=''):
        self.name = name
        self.email = email
❷    def introduce(self):
        print('我是 {}, 电子邮箱是: {}'.format(self.name, self.email))
❸ class Student(Person):
    def __init__(self, name, email, stu_no, class_name, courses):
        super().__init__(name, email)
        self.stu_no = stu_no
        self.class_name = class_name
        self.courses = courses.copy()
❹    def introduce(self):
        print('我是来自{}的学生, 学号{}'.format(self.class_name, self.stu_no))
❺ class Teacher(Person):
    def __init__(self, name, email, teacher_no, dept, course):
        super().__init__(name, email)
        self.teacher_no = teacher_no
        self.department = dept
        self.course = course
❻    def introduce(self):
        print('我是{}的老师, 教授{}课程'.format(self.department, self.course))
❼ wangwu = Person('王五', 'wangwu@example.com')
    wangwu.introduce()
❽ zhangsan = Student('张三', 'zhangsan@example.com', '1008', '十班', ['程序设计', '金融数据分析'])
    zhangsan.introduce()
❾ teacher_li = Teacher('李老师', 'li@example.com', 'T1001', '金融学院', '金融数据分析')
    teacher_li.introduce()
```

我是王五, 电子邮箱是: wangwu@example.com

我是来自 十班 的学生, 学号: 1008

我是金融学院的老师, 教授金融数据分析课程

课堂练习5： 动物园管理： 叫一声

场景： 动物园需要给所有动物上“疫苗”，管理员希望不关心具体动物类型，统一调用 `make_sound()` 方法。

任务：

- 定义父类 `Animal`，有方法 `make_sound(self)`，里面打印 "这是一只动物，它发出奇怪的声音"。
- 定义子类 `Dog`，重写 `make_sound`，打印 " 旺财！"。
- 定义子类 `Cat`，重写 `make_sound`，打印 " 喵喵喵~"。
- 定义一个函数 `vaccinate(animal)`，它接收一个动物对象，先打印 "打针前："，再调用 `animal.make_sound()`。
- 创建 `Dog()` 和 `Cat()` 实例，传给 `vaccinate` 函数。

封装

- 为保证类内容数据结构的完整性，在设计类时，一些属性和方法会“隐藏”起来。这些属性和方法不能通过“类对象.属性名”或者“类对象.方法名（参数）”的形式进行调用。这就是面向对象编程中的**封装**（encapsulation）。
- 由于被“隐藏”的属性和方法只能在类内部使用，因此称之为**私有属性和私有方法**。相对地，其他属性和方法被称为**公有属性和公有方法**。
- 为了实现类的封装，Python 采用以下方法：
 - 默认情况下，类中名称前面没有双下划线“__”的属性和方法都是公有（public）的。
 - 类中以下划线“_”或双下划线“__”开头的属性和方法都是私有（private）的。

封装

【例】定义 Stock 类，包含股票名称和价格，属性名称为 name 和 price。将 price 定义为私有属性，以保证价格大于 0。

property(fget=None, fset=None)

```
❶ class Stock:
❷     def __init__(self, p_name):
❸         self.name = p_name
❹
❺     def setprice(self, p_price):
❻         self.__price = max(0, p_price)
❼
❽     def getprice(self):
❾         return self.__price
❼
❽     price = property(getprice, setprice)
❹
❺ stock1 = Stock('中国银行')
❻ stock1.price = -10.00
❼ print(stock1.price)
❹
❺ stock1.price = 3.19
❼ print(stock1.price)
❹
❺
❻ 0
❼ 3.19
```

课堂练习6：电视遥控器

场景：设计一个 TV 类，音量必须在 0-100 之间，不能直接访问内部 `_volume`。

任务：

- 定义 TV 类，初始化 `self._volume = 50`。
- 定义 `set_volume(self, vol)`，如果 $0 \leq \text{vol} \leq 100$ 则修改，否则打印 "⚠️ 音量非法！耳膜破裂边缘！"。
- 定义 `get_volume(self)`，返回 `self._volume`。
- 用 `property` 把 `volume` 绑定到 `getter/setter`，让学生可以用 `tv.volume = 999` 体验“保护机制”。

Happy coding! 🐍



综合练习7：管理宠物

小明养了宠物，需要管理它们的信息。请设计一个类，并用列表存储宠物信息。

功能要求

1. Pet 类设计：

- 属性：名字(name)、品种(breed)、年龄(age)
- 方法：
 - `__init__(self, name, breed, age)` - 构造方法
 - `__str__(self)` - 返回宠物信息字符串（如："旺财（金毛），3岁"）
 - `is_puppy(self)` - 判断是否为幼宠（年龄<1岁返回True）

2. 主程序功能：

- 从文件 `pets.txt` 读取宠物信息
- 显示所有宠物列表
- 让用户输入新宠物信息，添加到列表
- 显示所有幼宠（年龄<1岁）
- 将宠物信息保存回文件

3. 数据文件格式示例：

`pets.txt` 内容：

旺财,金毛,3
咪咪,波斯猫,2
豆豆,泰迪,0.5

提示

- 使用 `split(',')` 解析文件中的每一行
- 使用 `input()` 获取用户输入
- 使用 `with open` 读写文件

综合练习8：待办事项管理器

开发一个简单的待办事项管理程序，帮助记录每天要做的事情。

功能要求

1. Task 类设计：

- 属性：标题(title)、描述(description)、是否完成(is_done)
- 方法：
 - `__init__(self, title, description="")` - 构造方法
 - `mark_done(self)` - 标记为已完成
 - `__str__(self)` - 返回任务字符串（已完成的任务前面加✓）

2. 主程序功能：

- 从文件 `tasks.txt` 加载任务
- 显示所有任务（区分已完成/未完成）
- 添加新任务
- 标记任务为已完成
- 删除任务
- 保存任务到文件

3. 数据文件格式示例：

`tasks.txt` 内容：

```
买菜,买今晚的食材,False  
写作业,完成数学作业,False  
还书,去图书馆还书,True
```

提示

- 使用布尔值记录任务完成状态
- 使用 `enumerate()` 获取任务序号
- 字符串转换为布尔：`"True"→True`,
`"False"→False`

综合练习9：学生信息管理

班主任需要一个简单的程序来管理班级学生的基本信息，包括添加学生、查找学生、统计男女生人数等功能。

功能要求

1. Student 类设计：

- 属性：学号(stu_id)、姓名(name)、性别(gender)、年龄(age)
- 方法：
 - `__init__(self, stu_id, name, gender, age)` - 构造方法
 - `__str__(self)` - 返回学生信息字符串
 - `to_string(self)` - 返回用于存储的字符串（用逗号分隔）

2. 主程序功能：

- 从文件 `students.txt` 加载学生信息
- 显示所有学生列表
- 按学号查找学生
- 按姓名查找学生
- 添加新学生
- 删除学生
- 统计男女生人数
- 保存数据到文件

3. 数据文件格式示例：students.txt 内容：

```
1001,张三,男,19
1002,李四,女,18
1003,王芳,女,20
```

提示

- 使用字典存储学生：`students[stu_id] = Student(...)`
- 按姓名查找时，可以模糊匹配（使用 `in` 关键字）